

Computer Lab: training a neural network

4AI11 : Optimization for Machine Learning

28 January 2026

Submission and grading information

You can do the computer lab alone or in pairs. Please write a report and post it on **e-campus**. You can do it as a jupyter notebook or a pdf file.

Only the fact that you produce a report counts in the final grade. **Note however that the final exam will contain multiple-choice questions related to the tools that you use in this computer lab.**

The started code is provided in a Jupyter notebook. To run the code, you will need to have **Pytorch** installed. If you have issues with this library on your computer, you can do the lab online on Google colab.

1 Getting started with Pytorch

In this lab, you will use the **Pytorch** library in Python. Pytorch is designed to facilitate and optimize training of large machine learning and deep learning models. It implements several key features:

- fast numerical linear algebra operations using parallelization (if a GPU is available),
- automatic differentiation, which allows to compute gradients numerically,
- tools for handling datasets, pre-processing and model evaluation.

Question 1.1

Do the Pytorch tutorial provided in Section 1 of the notebook. Observe the efficiency of GPU acceleration and try out automatic differentiation on a quadratic function.

2 Dataset: digit recognition

Our goal is to train a neural network to classify hand-written digits from the MNIST dataset. This is a very standard machine learning benchmark. The dataset can be readily downloaded using Pytorch: the code is provided in the notebook.

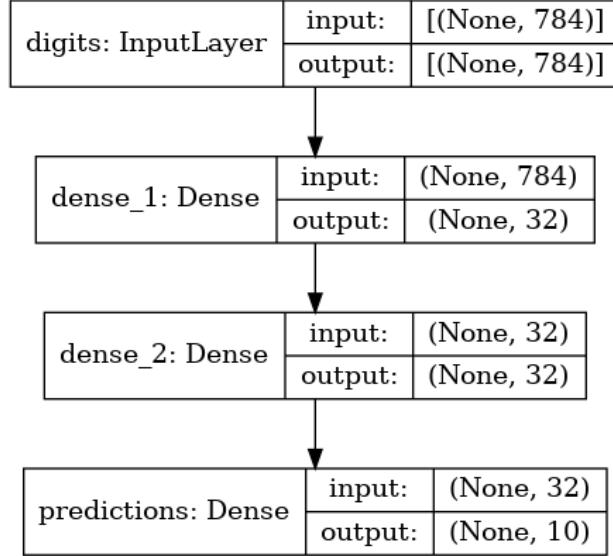


Figure 1: Architecture of the neural network.

3 Neural network model

Our goal is to predict, given an image, the digit it contains. It is therefore a classification problem with 10 classes.

We use a fully-connected neural network with three layers. Given an input image x , the neural network does a series of computations and returns a vector $\Phi_\theta(x) \in \mathbb{R}^{10}$, where θ denotes the neural network parameters, and the vector encodes the confidence scores associated by the neural network to each class. The predicted class corresponds to the index with the highest confidence score.

We train the neural network by solving the optimization problem

$$\min_{\theta} \frac{1}{n} \sum_{i=1}^n \ell(\Phi_\theta(x_i); y_i^{\text{true}}),$$

where n is the total number of training samples. The loss function ℓ is the categorical cross-entropy loss defined as

$$\ell(\Phi_\theta(x_i); y_i^{\text{true}}) = - \sum_{d=0}^9 y_{id}^{\text{true}} \log \left(\frac{[\Phi_\theta(x_i)]_d}{\sum_{d'=0}^9 [\Phi_\theta(x_i)]_{d'}} \right),$$

where $y_{id}^{\text{true}} = 1$ if image i represents digit d and $y_{id}^{\text{true}} = 0$ otherwise. This is a generalization of the logistic loss to problems with more than two classes.

Question 3.1

How many optimization variables are we going to train using this model?

4 Stochastic gradient descent

To train the model, we need to compute the gradient of the loss with respect to the parameters θ . Libraries such as Pytorch allow to compute the model gradient using automatic differentiation. This requires two steps:

1. A forward propagation step, where the loss is computed on the current batch, and the intermediate layer activations are stored in memory,
2. a backward propagation step, where the activations are used to compute the gradients via the chain rule.

An example code for computing the gradient of a batch of the data is provided.

We propose to implement minibatch stochastic gradient descent with step size sequence $\{\gamma_k\}$, which performs the iterations for $k = 0, 1, \dots$:

Select a batch b_{k+1} of samples of size B

$$\theta_{k+1} = \theta_k - \frac{\gamma_k}{B} \sum_{i \in b_{k+1}} \nabla \ell(\Phi_{\theta_k}(x_i), y_i^{\text{true}})$$

Question 4.1

Using the helper code, implement stochastic gradient descent.

Question 4.2

Run it for one pass over the data, also called one **epoch**. Plot the objective value as a function of the iterations. Compare different step size (fixed, decreasing...) and minibatch choices. What do you observe?

Remarks:

- Do not compute the objective value at each iteration, only from time to time (e.g., every 1000 iterations). This is sufficient for plotting purposes and will save considerable computing time.
- Instead of sampling one minibatch per iteration, it is much more common to pre-shuffle the data before each epoch, and take minibatches sequentially. Although this is not exactly what we described in theory, it is more efficient computationally, and ensures that each data point is seen exactly once per epoch.

5 Evaluation of the model

Question 5.1

Evaluate the accuracy of the models on the training set and on the test set.

Question 5.2

With the step size and batch size that you selected previously, run SGD for 5 epochs. Plot the evolution of the accuracy. What do you observe? How would you choose when to stop training?